

SafeCity Analytics: Predictive Crime Analysis of Los Angeles Using Machine Learning and Distributed Computing

Harsh Mahesh Tikone
University at Buffalo
EAS 587, Spring 2026

Dev Desai
University at Buffalo
EAS 587, Spring 2026

Shwetangi
University at Buffalo
EAS 587, Spring 2026

Abstract—This paper describes SafeCity Analytics, a four-phase data science project built on LAPD crime records from 2024 (62,105 incidents). We cleaned and explored the raw data in Phase 1, then trained six machine learning models in Phase 2 covering crime category prediction (13 classes), severity classification, geographic hotspot discovery, and weapon-involvement risk scoring. Random Forest and Decision Tree both reached around 81% accuracy on crime category; k-Nearest Neighbors hit 99.8% on severity, which is a derived label and not a surprising result. We deployed the Random Forest as a Model Context Protocol (MCP) server so it can be queried through Claude Code in natural language. Phase 3 moved the pipeline to Databricks, implementing a Bronze-Silver-Gold Delta Lake architecture and joining in a secondary FBI NIBRS dataset for cross-source validation. The Spark MLlib Logistic Regression weapon predictor reached ROC-AUC of 0.9123, which was better than the Phase 2 sklearn baseline. The main takeaways are that crime type is largely predictable from location, time, and premise; weapon prediction works best as a risk score rather than a hard label; and LA crime geography splits cleanly into two geographic clusters.

Index Terms—crime prediction, machine learning, Apache Spark, model context protocol, random forest, k-means, LAPD, Delta Lake, data engineering

I. INTRODUCTION AND BACKGROUND

Most crime prediction tools deployed today are dashboards they show what happened, not what is likely to happen next. This limits their value for officers in the field or planners trying to allocate resources ahead of time [1]. SafeCity Analytics was built to close that gap: starting from raw LAPD records, we built a pipeline that ends with a live inference server callable through natural language via the Model Context Protocol [9].

The LAPD publishes crime incident data on the Los Angeles Open Data portal [2]. The 2024 extract we used has 62,105 rows and 28 columns, covering crime type, location, time, victim demographics, and weapon use.

We had three concrete questions driving the work. First: given time, location, and context, what type of crime is this? Answering this could help with dispatch pre-filling and officer prep. Second: does this incident carry elevated weapon risk? A calibrated probability is more useful than a binary flag for officer safety decisions. Third: which geographic areas have the highest crime density, and does that pattern hold over time?

Related systems like PredPol and ShotSpotter apply spatial-temporal models for patrol guidance [1]. We drew on ensemble

methods from the ML literature for multi-class crime classification [4]. What sets this project apart is the cross-source validation with FBI NIBRS data [3] and the deployment of a model through the MCP architecture [9] so it can be queried conversationally.

II. DATA DESCRIPTION

A. Primary Dataset

We used the LAPD Crime Data from 2020 to Present, filtered to calendar year 2024 [2]. The raw file has 62,105 rows and 28 columns. Each record includes the crime code and description, LAPD area (1–21), GPS coordinates, victim age, sex, and descent, premise type, weapon code, and case status.

B. Secondary Dataset

For Phase 3 we brought in two FBI NIBRS tables for 2024 [3]: a Victims table (232,660 rows) and an Offenses table (250,127 rows). We ingested both as Bronze Delta tables and left-joined them to the LAPD base on CaseNo. The join rate was poor: only 45 of 62,105 rows (0.1%) matched, because LAPD DR_NO and NIBRS CaseNo use different ID formats. Rather than treating this as a failure, we analyzed the NIBRS tables independently and compared distributions to validate findings across both sources.

C. Data Pipeline Overview

Fig. 1 illustrates the end-to-end data flow.

D. Data Cleaning

We applied ten cleaning steps in `src/data_cleaning.py`: (1) date columns converted to datetime with Year, Month, Day, DayOfWeek, Hour extracted; (2) invalid TIME_OCC values (>2400) fixed; (3) victim age of 0 replaced with NaN (it means unknown, not infant); (4) sex and descent codes standardized; (5) columns Crm Cd 2/3/4 dropped since they were 98–100% missing; (6) 140+ crime descriptions collapsed into 12 categories; (7) premise types grouped into 9 categories; (8) victim ages binned into 7 groups; (9) coordinates checked against the LA bounding box (no outliers found); (10) reporting delay computed as days between occurrence and report date. After cleaning, the dataset has 62,105 rows and 46 features.

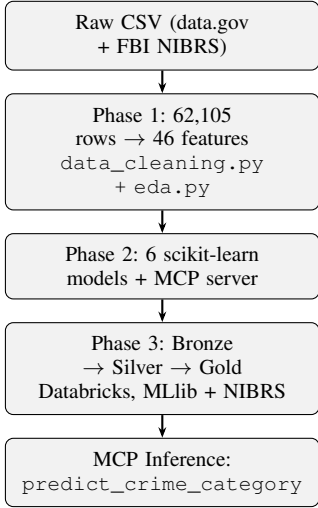


Fig. 1. End-to-end SafeCity Analytics data flow.

E. Key EDA Findings

We followed Tukey’s exploratory principles [6]. A few findings shaped the modeling decisions downstream. Crime peaks at 18:00 with a secondary spike at 08:00. Vehicle crime is dominant at 46.2% of all incidents. Weapon involvement is lower than expected: only 5.9%, with strong-arm (physical force, no weapon) being the most common type. Nearly half the records (48.5%) have unknown victim demographics, mostly from property crimes where no victim is present at the scene. Three-quarters of crimes were reported within 3 days. Central LA has the most incidents (6,024); Foothill the fewest (1,774).

III. METHODOLOGY

A. Feature Engineering

To avoid data leakage from inconsistent per-script encoding, we built a shared module (`src/models/preprocess.py`) that all six models pull from. All random seeds are fixed at 42. The ten features are in Table I.

TABLE I
FEATURE SET FOR SUPERVISED MODELS

Feature	Type	Description
AREA	Numeric	LAPD area code (1–21)
Hour	Numeric	Hour of occurrence (0–23)
Month	Numeric	Month (1–12)
IsWeekend	Binary	1 if Saturday or Sunday
Has Weapon	Binary	1 if weapon was used
Premise Cat.	Encoded	9-class categorical
TimeBucket	Encoded	Morning/Afternoon/Evening/Night
Severity	Encoded	High / Medium / Low
Part 1-2	Numeric	LAPD seriousness class
Reporting Delay	Numeric	Days from crime to report

TABLE II
ML ALGORITHM SUMMARY

Algorithm	Class	Target	Key Detail
kNN	In-class	Severity (3)	k=3, scaled
Decision Tree	In-class	Category (13)	GridSearchCV
k-Means	In-class	Hotspots	k=2, silhouette
Naive Bayes	In-class	Category (13)	ComplementNB
Random Forest	Outside	Category + MCP	Balanced weights
Logistic Reg.	Outside	Weapon (binary)	L2, prob. cal.

B. Machine Learning Models

We trained six algorithms; Table II shows the setup.

k-Nearest Neighbors predicts crime severity (High/Medium/Low). We scaled features with `StandardScaler` since kNN is distance-based. We ran 5-fold cross-validation over $k \in \{3, 5, 7, 9, 11, 13, 15\}$ and $k = 3$ came out best.

Decision Tree predicts the 13-class crime category. We tuned `max_depth` and `min_samples_split` with `GridSearchCV`; best `max_depth` was 15.

k-Means finds geographic hotspots using latitude and longitude. We picked k by combining the elbow method with silhouette score, landing at $k = 2$ (silhouette = 0.546).

Naive Bayes uses `ComplementNB`. We chose it over `GaussianNB` through cross-validation because it handles the class imbalance better. Vehicle Crime makes up 46.2% of the data.

Random Forest predicts crime category as an ensemble of trees, which reduces overfitting compared to a single Decision Tree. We used `class_weight='balanced'` to handle class imbalance and tuned with `RandomizedSearchCV`. This model is also the one we deployed as the MCP server.

Logistic Regression predicts weapon involvement (True/False). We tuned regularization strength C via cross-validation. The key reason we used LR here rather than a tree model is that it outputs calibrated probabilities useful for threshold-based risk scoring in the field.

C. Spark and Delta Lake Architecture

Phase 3 moved the pipeline to Databricks using a three-layer Delta Lake structure across eight notebooks.

Bronze is raw ingestion only no transforms, just the data as-is with `_source` and `_ingested_at` audit columns added.

Silver is where the real work happens. All three Bronze tables get cleaned, type-cast, and left-joined on `CaseNo` into `silver_lapd_crimes` (62,105 rows, partitioned by AREA). We also derived six new features here: Hour, Month, DayOfWeek, IsWeekend, Reporting_Delay, and Has_Weapon.

Gold contains six pre-aggregated Delta tables covering crime counts by area, time, and category ready for dashboards or downstream models.

Mllib: Notebook 04 re-implements Decision Tree and Naive Bayes on crime category in Spark. Notebook 08 trains a Logistic Regression pipeline (`VectorAssembler` → `StandardScaler` → `LogisticRegression`) on weapon prediction using the NIBRS-augmented Silver features; the trained pipeline is saved to `part2_phase3_databrick_model/`.

We ran into two platform constraints worth noting. StringIndexer is blocked on Databricks Community Edition Shared clusters via the Py4J whitelist, so we encoded categories using Spark SQL `dense_rank()` window functions instead same result, different path. CrossValidator caused the CE node to run out of memory, so we used fixed hyperparameters (`regParam=0.01`, `maxDepth=15`) and deleted model objects from memory between notebook cells.

D. MCP Server Deployment

The trained Random Forest is served as an MCP server [9] via `src/mcp/server.py`. It exposes three tools: `predict_crime_category`, `list_crime_categories`, and `server_health`. If the model file is missing (e.g., Git LFS issues), the server retrains automatically from the cleaned CSV.

IV. RESULTS AND ANALYSIS

A. Phase 2: Classification Performance

Table III summarizes test-set performance. All models use an 80/20 stratified split with random seed 42.

TABLE III
PHASE 2 MODEL PERFORMANCE SUMMARY

Algorithm	Target	Acc.	$F1_w$	CV Acc.
kNN ($k = 3$)	Severity	0.998	0.998	
Decision Tree	Category	0.8114	0.7997	0.801
Random Forest	Category	0.815	0.806	0.814
Naive Bayes	Category	0.510	0.396	0.506
Logistic Reg.	Weapon	0.867	0.893	
k-Means	Clusters	Silhouette = 0.546, $k = 2$		

kNN hit 99.8% accuracy on severity. This is not surprising severity is computed directly from the LAPD Part 1-2 field, which is already in the feature set.

Decision Tree reached 81.1% accuracy across 13 crime categories. The strongest features were Premise Category (≈ 0.47) and Severity (≈ 0.29). Where a crime happens turns out to be the best single predictor of what type of crime it is. Minority classes like Homicide and Arson have fewer than 20 test samples and show $F1 \approx 0$ not much the model can do there.

Random Forest matched the Decision Tree at 81.5% accuracy with slightly better weighted F1 (0.806 vs. 0.800). Training took $10\times$ longer (3.15s vs. 0.32s), which is fine for a deployed server but worth noting.

Naive Bayes reached only 51.0% accuracy. The conditional independence assumption is the problem AREA and Premise Category are correlated, and that correlation matters a lot for distinguishing crime types.

Logistic Regression got ROC-AUC = 0.886 for weapon prediction. Precision on the weapon class is only 0.258, which is expected given the 5.9% base rate. The probability score is more useful operationally than a binary label.

Random Forest matched Decision Tree accuracy (81.5%) with marginally better weighted F1 (0.806 vs. 0.810), confirming the ensemble reduces overfitting. Training time was $10\times$ longer (3.15s vs. 0.32s), an acceptable trade-off for the deployed inference server.

B. Clustering Results

k-Means with $k = 2$ gave silhouette = 0.546. Cluster 0 is the coastal and west side (West LA, Pacific, Hollywood); Cluster 1 is the inland and central areas (Central, Newton, 77th Street). Vehicle crime is the top category in both clusters at 44–48%.

We tried DBSCAN first but it labeled more than 60% of points as noise. LA crime is spread too evenly across the city for DBSCAN to find tight clusters. k-Means made more sense here.

C. Phase 3: Spark MLlib Results

Table IV compares Phase 2 sklearn and Phase 3 MLlib results.

TABLE IV
PHASE 2 VS PHASE 3 MODEL COMPARISON

Algorithm	Metric	Phase 2	Phase 3
Decision Tree	Acc / F1	0.8114 / 0.7997	0.529 / 0.504
Naive Bayes	Acc / F1	0.510 / 0.396	0.397 / 0.306
Logistic Reg.	ROC-AUC	0.886	0.912
Logistic Reg.	Accuracy	0.867	0.939
Logistic Reg.	F1	0.893	0.921

Decision Tree and Naive Bayes both dropped in Phase 3. Three things explain this: Spark’s distributed shuffling changes the data order versus in-memory pandas; we couldn’t run cross-validation due to OOM on the CE node; and the split was 80/20 in MLlib vs. 60/20/20 in sklearn. The Logistic Regression weapon predictor actually improved, reaching ROC-AUC of 0.9123 on the NIBRS-augmented Silver features.

D. NIBRS Cross-Dataset Insights

Insight 1: Weapon crime is concentrated, not random. Pacific division has a 20.37% weapon rate (913 of 4,481 crimes), more than double the next highest area (Southeast at 9.36%). NIBRS breaks this down further: most offenses are property crimes (137,367), but it’s person-targeted crimes (85,820) that carry most of the weapon involvement.

Insight 2: Both sources agree on when crime happens. When we normalized LAPD and NIBRS hourly counts to percentage share, the curves are nearly identical: both dip between 5–6am and climb to an evening peak. Having two independent sources tell the same story gives us more confidence in using time of day as a feature.

Insight 3: Victim demographics and reporting delays. 83.6% of NIBRS victims are individuals; 13.1% are businesses. On the LAPD side, 48.5% of victim demographic fields are unknown. The reporting delay data is interesting: financial crimes like embezzlement take an average of 31 days to get reported, identity theft 18 days. Violent crimes are reported within a day or two.

V. DISCUSSION

A. Findings Interpretation

The clearest finding is that crime category is predictable at around 81% accuracy using just area, time of day, and premise type. That tells us most of the variance is structural it follows land use patterns and predictable human behavior rather than being random. This is consistent with what the feature importance plot shows: where a crime happens (Premise Category, ≈ 0.47) matters more than when.

Weapon prediction is harder. ROC-AUC of 0.912 means the model ranks incidents by risk well, but with only a 5.9% base rate, any fixed threshold is going to generate a lot of false positives. The right way to use this model is as a risk score that officers can act on above a tunable threshold, not as a binary prediction.

The geographic result was the most clear-cut. LA crime splits into two stable clusters: coastal/west and central/inland. Trying $k > 5$ didn't improve the silhouette score, which suggests the existing 21-division LAPD structure already captures the meaningful geographic variation.

B. Limitations

We only have one year of data (2024), so we can't look at seasonal patterns or year-over-year trends. Nearly half the victim demographic records are unknown, which limits what we can say about who is being victimized. The NIBRS join rate was 0.1%, which means the NIBRS enrichment columns in the Silver table are mostly empty and weren't useful for the joined dataset, so we had to rely on the independent analysis instead. We also haven't tested how the MCP server performs under concurrent load.

C. Lessons Learned

Building `preprocess.py` early was the right call. Having one shared encoding module meant we never had to debug subtle differences between how scripts handled the same features. When we moved to Spark in Phase 3, we could reproduce the same transformations without second-guessing what Phase 2 had done.

The DBSCAN failure was informative rather than wasted time: the 60%+ noise rate told us clearly that LA crime is spatially diffuse, which influenced how we interpreted the k-Means result. And the StringIndexer block on CE Shared clusters was the biggest surprise of Phase 3; we lost a few hours diagnosing it before finding the `dense_rank()` workaround.

VI. CONCLUSION

We set out to build a crime prediction pipeline on public LAPD data and ended up with something that works end-to-end: from raw CSV through cleaning and EDA, to six ML models, a deployed MCP inference server, and a distributed Spark implementation with cross-source NIBRS validation.

The main results: Decision Tree and Random Forest predict crime categories at around 81% accuracy; Logistic Regression scores weapon risk at ROC-AUC = 0.912 in Phase 3; k-Means

finds a stable two-cluster geographic structure at silhouette = 0.546.

Three things we'd do next: bring in multiple years of LAPD data to enable seasonal analysis; replace the flat 13-class label scheme with a hierarchical taxonomy; and connect the MCP server to a live LAPD data feed via Spark Structured Streaming for near-real-time inference.

REFERENCES

- [1] W. L. Perry, B. McInnis, C. C. Price, S. Smith, and J. S. Hollywood, *Predictive Policing: The Role of Crime Forecasting in Law Enforcement Operations*. Santa Monica, CA: RAND Corporation, 2013.
- [2] Los Angeles Police Department, "Crime Data from 2020 to Present," City of Los Angeles Open Data Portal, 2024. [Online]. Available: <https://data.lacity.org/>
- [3] Federal Bureau of Investigation, "National Incident-Based Reporting System (NIBRS)," FBI Crime Data Explorer, 2024. [Online]. Available: <https://cde.ucr.cjis.gov/>
- [4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] D. W. Hosmer and S. Lemeshow, *Applied Logistic Regression*, 2nd ed. New York, NY: Wiley, 2000.
- [6] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
- [7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [8] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [9] Anthropic, "Model Context Protocol Documentation," 2024. [Online]. Available: <https://modelcontextprotocol.io/>
- [10] J. VanderPlas, *Python Data Science Handbook*. Sebastopol, CA: O'Reilly, 2016.

APPENDIX

A. Shared Preprocessing Module

```
def get_classification_features(df):
    encoders = {}
    feature_names = [
        "AREA", "Hour", "Month",
        "IsWeekend", "Has_Weapon",
        "Part_1-2", "Reporting_Delay_(Days)"
    ]
    for col in ["Premise_Category",
               "TimeBucket", "Severity"]:
        le = LabelEncoder()
        df[col+"_enc"] = le.fit_transform(
            df[col].fillna("Unknown"))
        encoders[col] = le
        feature_names.append(col+"_enc")
    X = df[feature_names].fillna(0).values
    y = df["Crime_Category"].values
    return X, y, None, feature_names, encoders
```

B. Spark Bronze Ingestion

```
df_raw = (spark.read.format("csv")
           .option("header", "true")
           .option("inferSchema", "true")
           .load("/FileStore/lapd_crime_2024.csv"))

df_raw.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("bronze_mydata")

print(f"Bronze_rows:_{df_raw.count():,}")
```

C. MCP Tool Handler

```
@server.call_tool()
async def call_tool(name, arguments):
    if name == "predict_crime_category":
        features = _build_features(
            arguments, bundle)
        pred = bundle["model"].predict(
            [features])[0]
        proba = bundle["model"]\
            .predict_proba([features])[0]

        top3 = sorted(
            zip(bundle["model"].classes_,
                proba),
            key=lambda x: -x[1][:3])
        return [types.TextContent(
            type="text",
            text=(f"Predicted:_{pred}\n" +
                  "\n".join(
                      f"_{c}:_{p:.1%}"
                      for c, p in top3)))]
```